

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



VIEWMODEL DAN DEBUGGING

Oleh:

Randy Febrian

NIM. 2310817110013

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel dan Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Randy Febrian
NIM : 2310817110013

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code.....	7
B. Output Program	14
C. Pembahasan	17
SOAL 2.....	21
A. Penjelasan	21
TAUTAN GIT	22

DAFTAR GAMBAR

Gambar 1. Screenshot Home Screen Soal 1	14
Gambar 2. Screenshot Detail Screen Soal 1	14
Gambar 3. Screenshot Browser Setelah Intent Soal 1	15
Gambar 4. Penggunaan Debugger	15
Gambar 5. Tampilan log di logcat	16

DAFTAR TABEL

Tabel 1. Jawaban Soal 1 MainActivity.kt	7
Tabel 2. Jawaban Soal 1 HomeScreen.kt	7
Tabel 3. Jawaban Soal 1 DetailScreen.kt	9
Tabel 4. Jawaban Soal 1 ItemData.kt	11
Tabel 5. Jawaban Soal 1 Navigation.kt	11
Tabel 6. Jawaban Soal 1 ItemViewModel.kt.....	12
Tabel 7. Jawaban Soal 1 ViewModelFactory.kt.....	13

SOAL 1

Soal Praktikum

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory dalam pembuatan ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. gunakan logging untuk event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
- e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out fitur Step Into, Step Over, dan Step Out

A. Source Code

Tabel 1. Jawaban Soal 1 MainActivity.kt

```

1 package com.example.scrollablelist
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.activity.enableEdgeToEdge
7 import androidx.lifecycle.viewmodel.compose.viewModel
8
9 class MainActivity : ComponentActivity() {
10     override fun onCreate(savedInstanceState: Bundle?)
11     {
12         super.onCreate(savedInstanceState)
13         enableEdgeToEdge()
14         setContent {
15             val viewModel: ItemViewModel =
16             viewModel(factory = ItemViewModelFactory())
17             Navigation()
18         }
19     }
20 }

```

Tabel 2. Jawaban Soal 1 HomeScreen.kt

```

1 package com.example.scrollablelist
2
3 import android.content.Intent
4 import android.net.Uri
5 import androidx.compose.foundation.Image
6 import androidx.compose.foundation.layout.*
7 import androidx.compose.foundation.lazy.LazyColumn
8 import androidx.compose.foundation.lazy.items
9 import
10 androidx.compose.foundation.shape.RoundedCornerShape
11 import androidx.compose.material3.Button
12 import androidx.compose.material3.Card
13 import androidx.compose.material3.MaterialTheme
14 import androidx.compose.material3.Text
15 import androidx.compose.runtime.Composable
16 import androidx.compose.runtime.collectAsState
17 import androidx.compose.ui.Alignment
18 import androidx.compose.ui.Modifier
19 import androidx.compose.ui.draw.clip
20 import androidx.compose.ui.platform.LocalContext
21 import androidx.compose.ui.res.painterResource
22 import androidx.compose.ui.layout.ContentScale

```

```

23 import androidx.compose.ui.res.stringResource
24 import androidx.compose.ui.unit.dp
25 import androidx.navigation.NavController
26
27 @Composable
28 fun HomeScreen(navController: NavController, viewModel:
29 ItemViewModel) {
30     val context = LocalContext.current
31     val itemList = viewModel.itemList.collectAsState()
32     Column(modifier = Modifier
33         .fillMaxSize()
34         .padding(16.dp)) {
35         LazyColumn {
36             items(itemList.value) { item ->
37                 Card(
38                     modifier = Modifier
39                         .padding(8.dp)
40                         .fillMaxWidth()
41                 ) {
42                     Row(modifier =
43 Modifier.padding(16.dp)) {
44                         Image(
45                             painter =
46 painterResource(id = item.pictureId),
47                             contentDescription = null,
48                             contentScale =
49 ContentScale.Crop,
50                             modifier = Modifier
51                                 .height(130.dp)
52                                 .width(100.dp)
53
54 .align(Alignment.CenterVertically)
55
56 .clip(RoundedCornerShape(10.dp))
57                             )
58                         Column(modifier = Modifier
59                             .padding(start = 16.dp)
60                             .fillMaxWidth()) {
61
62                             Text(text = item.name,
63 style = MaterialTheme.typography.titleMedium)
64                             Text(text =
65 stringResource(id = item.description), style =
66 MaterialTheme.typography.bodyMedium)
67
68                             Row(
69                                 modifier = Modifier

```



```

70                                     .padding(top =
71 8.dp)
72                                     .fillMaxWidth(),
73                                     horizontalArrangement =
74 Arrangement.SpaceBetween
75                                     ) {
76     Button(onClick = {
77
78 android.util.Log.d("HomeScreen", "Tombol Detail ditekan
79 untuk ${item.name}")
80
81 viewModel.setSelectedItem(item)
82
83 navController.navigate("detail_Screen")
84                                     }) {
85     Text("Detail")
86                                     }
87
88     Button(onClick = {
89
90 android.util.Log.d("HomeScreen", "Tombol Explicit
91 Intent ditekan: ${item.url}")
92                                     val intent =
93 Intent(Intent.ACTION_VIEW, Uri.parse(item.url))
94
95 context.startActivity(intent)
96                                     }) {
97     Text("Buka Situs")
98                                     }
99     }
100 }
101 }
102 }
103 }
104 }
105 }
106 }

```

Tabel 3. Jawaban Soal 1 DetailScreen.kt

```

1 package com.example.scrollablelist
2
3 import android.util.Log
4 import androidx.compose.foundation.Image
5 import androidx.compose.foundation.layout.*
6 import androidx.compose.material3.MaterialTheme
7 import androidx.compose.material3.Text
8 import androidx.compose.runtime.Composable
9 import androidx.compose.runtime.collectAsState

```

```

10 import androidx.compose.runtime.getValue
11 import androidx.compose.ui.Alignment
12 import androidx.compose.ui.Modifier
13 import androidx.compose.ui.layout.ContentScale
14 import androidx.compose.ui.res.painterResource
15 import androidx.compose.ui.res.stringResource
16 import androidx.compose.ui.unit.dp
17 import androidx.navigation.NavController
18
19 @Composable
20 fun DetailScreen(viewModel: ItemViewModel,
21 navController: NavController) {
22     val selectedItem by
23     viewModel.selectedItem.collectAsState()
24
25     selectedItem?.let { itemData ->
26         Log.d("DetailScreen", "Menampilkan detail
27 untuk: ${itemData.name}")
28
29         Column(
30             Modifier
31                 .fillMaxSize()
32                 .padding(16.dp)
33         ) {
34             Image(
35                 painter = painterResource(id =
36 itemData.pictureId),
37                 contentDescription = null,
38                 contentScale = ContentScale.Crop,
39                 modifier = Modifier
40                     .fillMaxWidth()
41                     .padding(top = 10.dp)
42             )
43             Text(
44                 text = itemData.name,
45                 style =
46 MaterialTheme.typography.titleLarge,
47                 modifier = Modifier
48                     .padding(vertical = 10.dp)
49
50 .align(Alignment.CenterHorizontally)
51             )
52             Text(
53                 text = stringResource(id =
54 itemData.description),
55                 style =
56 MaterialTheme.typography.bodyMedium

```

```

57         )
58     }
59     } ?: run {
60         Text("Data tidak ditemukan", modifier =
61 Modifier.padding(16.dp))
62     }
63 }

```

Tabel 4. Jawaban Soal 1 ItemData.kt

```

1 package com.example.scrollablelist
2
3 import android.os.Parcelable
4 import kotlinx.parcelize.Parcelize
5
6 @Parcelize
7 data class ItemData(
8     val name: String,
9     val pictureId: Int,
10    val url: String,
11    val description: Int
12 ) : Parcelable

```

Tabel 5. Jawaban Soal 1 Navigation.kt

```

1 package com.example.scrollablelist
2
3 import androidx.compose.runtime.Composable
4 import androidx.lifecycle.viewmodel.compose.viewModel
5 import androidx.navigation.compose.NavHost
6 import androidx.navigation.compose.composable
7 import
8 androidx.navigation.compose.rememberNavController
9
10 @Composable
11 fun Navigation() {
12     val navController = rememberNavController()
13
14     val viewModel: ItemViewModel = viewModel(
15         factory = ItemViewModelFactory()
16     )
17
18     NavHost(navController = navController,
19 startDestination = "home_Screen") {
20         composable("home_Screen") {
21             HomeScreen(navController, viewModel)
22         }
23         composable("detail_Screen") {
24             DetailScreen(viewModel = viewModel,
25 navController = navController)
26         }
27     }
28 }

```

27	}
28	}

Tabel 6. Jawaban Soal 1 ItemViewModel.kt

1	package com.example.scrollablelist
2	
3	import android.util.Log
4	import androidx.lifecycle.ViewModel
5	import kotlinx.coroutines.flow.MutableStateFlow
6	import kotlinx.coroutines.flow.asStateFlow
7	
8	class ItemViewModel : ViewModel() {
9	
10	private val _itemList =
11	MutableStateFlow<List<ItemData>>(emptyList())
12	val itemList = _itemList.asStateFlow()
13	
14	private val _selectedItem =
15	MutableStateFlow<ItemData?>(null)
16	val selectedItem = _selectedItem.asStateFlow()
17	
18	init {
19	val itemList = listOf(
20	ItemData("Cirrostratus",
21	R.drawable.cirrostratus,
22	"https://id.wikipedia.org/wiki/Awan_sirostratus",
23	R.string.cirrostratus_desc),
24	ItemData("Cirrocumulus",
25	R.drawable.cirrocumulus,
26	"https://id.wikipedia.org/wiki/Awan_sirokumulus",
27	R.string.cirrocumulus_desc),
28	ItemData("Cumulus", R.drawable.cumulus,
29	"https://id.wikipedia.org/wiki/Awan_kumulus",
30	R.string.cumulus_desc),
31	ItemData("Altostratus",
32	R.drawable.altostratus,
33	"https://id.wikipedia.org/wiki/Awan_altostratus",
34	R.string.altostratus_desc),
35	ItemData("Cumulonimbus",
36	R.drawable.cumulonimbus,
37	"https://id.wikipedia.org/wiki/Awan_kumulonimbus",
38	R.string.cumulonimbus_desc),
39	)
40	
41	_itemList.value = itemList
42	Log.d("ItemViewModel", "Item list initialized
43	with \${itemList.size} items")
44	

```

45         itemList.forEach { item ->
46             Log.d("ItemViewModel", "Item dimuat:
47 ${item.name}")
48         }
49     }
50
51     fun setSelectedItem(item: ItemData) {
52         _selectedItem.value = item
53         Log.d("ItemViewModel", "Selected item:
54 ${item.name}")
55     }
56 }

```

Tabel 7. Jawaban Soal 1 ViewModelFactory.kt

```

1 package com.example.scrollablelist
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.ViewModelProvider
5
6 class ItemViewModelFactory : ViewModelProvider.Factory {
7     override fun <T : ViewModel> create(modelClass:
8 Class<T>): T {
9         if
10 (modelClass.isAssignableFrom(ItemViewModel::class.java))
11 {
12             return ItemViewModel() as T
13         }
14         throw IllegalArgumentException("Unknown
15 ViewModel class")
16     }
17 }

```

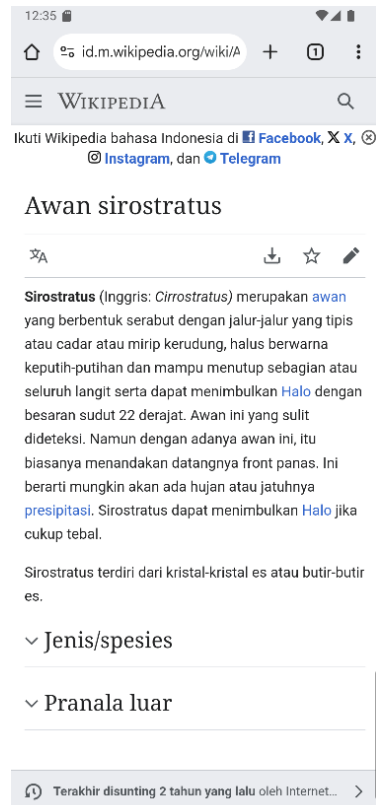
B. Output Program



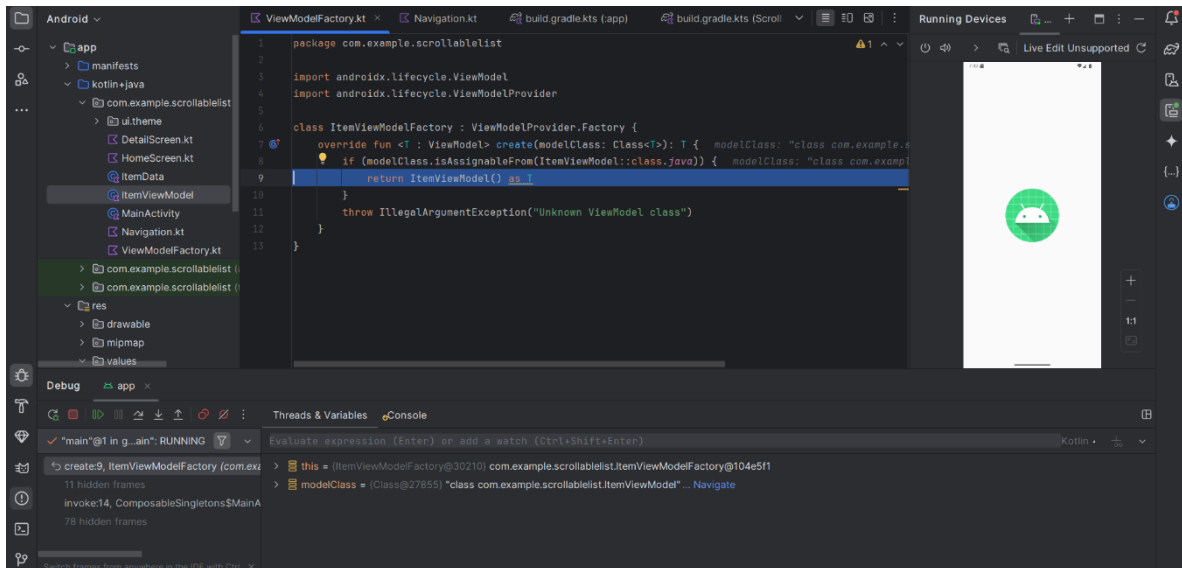
Gambar 1. Screenshot Home Screen Soal 1



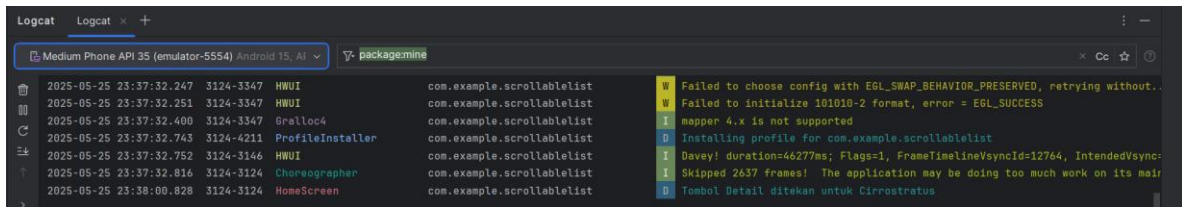
Gambar 2. Screenshot Detail Screen Soal 1



Gambar 3. Screenshot Browser Setelah Intent Soal 1



Gambar 4. Penggunaan Debugger



Gambar 5. Tampilan log di logcat

C. Pembahasan

HomeScreen.kt

Pada baris [1] dengan deklarasi `package com.example.scrollablelist` yang menunjukkan bahwa file ini merupakan bagian dari struktur aplikasi Android tersebut. Selanjutnya, pada baris [3 – 24] berfungsi untuk mengimpor beberapa library untuk UI menggunakan Jetpack Compose, seperti `Intent` dan `Uri` digunakan untuk membuka URL eksternal, sedangkan komponen seperti `Column`, `Row`, `LazyColumn`, `Image`, `Card`, dan `Button` dari Jetpack Compose digunakan untuk menyusun dan menampilkan antarmuka pengguna. Library `painterResource` dan `stringResource` juga diimpor agar gambar dan teks dari folder `res` dapat digunakan dalam komposisi UI.

Pada baris [27 - 28], berfungsi untuk menampilkan halaman utama dari aplikasi. Fungsi ini menerima dua parameter, yaitu `NavController` untuk mengatur navigasi antar layar, dan `viewModel` untuk menyimpan serta mengelola data item. Pada baris [29] berfungsi untuk mengambil konteks aplikasi menggunakan `LocalContext.current`, yang nantinya digunakan untuk menjalankan aktivitas seperti membuka browser eksternal.

Selanjutnya, pada baris [30 - 32] berfungsi untuk membuat sebuah `Column` sebagai layout utama yang mengisi seluruh ukuran layar dan diberi `padding` sebesar 16dp. Kemudian, baris [33] berfungsi untuk membuat `LazyColumn` yang digunakan untuk menampilkan daftar item secara efisien dan dapat `discroll`. Kemudian, baris [34 - 39], membuat `items(itemList)` setiap item dari daftar `itemList` akan ditampilkan satu per satu dengan komponen `Card`.

Baris [40 - 41] berfungsi untuk membuat `Row` yang digunakan untuk menyusun elemen secara horizontal, yaitu gambar di sebelah kiri dan informasi item di sebelah kanan. Pada baris [42 – 55] berfungsi untuk menampilkan gambar menggunakan komponen `Image`, dengan ukuran tinggi 130dp dan lebar 100dp serta diberi `rounded corner` menggunakan `clip(RoundedCornerShape(10.dp))`.

Pada baris [56 – 73] berfungsi untuk menampilkan informasi nama dan deskripsi item ditampilkan menggunakan `Text`. Di bawah informasi tersebut, terdapat dua tombol aksi yang diletakkan dalam satu `Row` dengan pengaturan `horizontalArrangement = Arrangement.SpaceBetween` agar kedua tombol berada di kiri dan kanan secara merata. Baris [48 – 73], terdapat `Row` yang memuat dua tombol aksi, yaitu tombol “Detail” dan “Buka Situs”, yang diatur dengan `Arrangement.SpaceBetween` agar berjauhan secara merata.

Tombol “Detail” didefinisikan pada baris [49 – 56], yang saat ditekan akan mencatat log di logcat, menyimpan item yang dipilih ke dalam viewModel dengan `setSelectedItem(item)`, lalu menavigasi ke layar detail menggunakan `navController.navigate("detail_Screen")`. Sementara itu, tombol “Buka Situs” pada baris [57 – 64] akan membuka browser eksternal dengan `Intent.ACTION_VIEW` menuju URL yang berasal dari properti `item.url`.

DetailScreen.kt

Baris [1] berfungsi sebagai penanda bahwa `DetailScreen.kt` merupakan bagian dari package `com.example.scrollablelist`. Baris [3 – 17] berfungsi untuk mengimpor berbagai library dari Jetpack Compose yang dibutuhkan untuk membangun antarmuka pengguna. Library tersebut digunakan untuk menampilkan gambar dan teks, mengatur layout vertikal, serta mengambil sumber daya dari folder `res`, seperti gambar dan string.

Baris [20 – 21] berfungsi untuk mendeklarasikan fungsi `DetailScreen` yang digunakan sebagai tampilan halaman detail dari item yang dipilih. Fungsi ini menerima dua parameter, yaitu `itemData` yang berisi data dari item yang akan ditampilkan, dan `navController` yang digunakan untuk navigasi antar halaman.

Baris [] digunakan untuk mengambil data item yang dipilih dari `ItemViewModel` menggunakan `collectAsState`, dan mengecek apakah data tersebut tidak null. Jika data tersedia, maka akan ditampilkan detailnya; jika tidak, akan muncul teks fallback. Baris [25] menambahkan log menggunakan `Log.d` yang mencetak nama item yang ditampilkan ke logcat, berguna untuk keperluan debugging.

Baris [22 – 31] berfungsi untuk membuat layout vertikal menggunakan `Column`. Di dalamnya, gambar dari item ditampilkan di bagian atas layar dengan ukuran yang disesuaikan dengan lebar layar. Gambar tersebut ditampilkan dengan tampilan yang sudut yang melengkung.

Baris [32 - 36] berfungsi untuk menampilkan nama item yang diposisikan di tengah layar dengan ukuran teks yang lebih menonjol agar mudah terlihat oleh pengguna. Baris [37 - 40] berfungsi untuk menampilkan deskripsi dari item dalam bentuk teks biasa. Teks ini diambil dari file sumber daya aplikasi dan ditampilkan tepat di bawah nama item.

ItemData.kt

Baris [1] menunjukkan bahwa file ini merupakan bagian dari package `com.example.scrollablelist`. Baris [3 – 4] mengimpor library yang diperlukan untuk membuat

kelas data yang dapat dikirim antar komponen Android menggunakan mekanisme Parcelable. Baris [6] menggunakan anotasi `@Parcelize` untuk menyederhanakan proses implementasi Parcelable secara otomatis tanpa perlu menulis kode tambahan.

Baris [7 – 12] mendefinisikan kelas data `ItemData` yang berisi struktur data untuk tiap item dalam aplikasi. Kelas ini digunakan untuk menyimpan dan mengelola data yang akan ditampilkan pada antarmuka pengguna serta memudahkan pengiriman data antar layar.

Navigation.kt

Baris [1] berfungsi sebagai penanda bahwa `Navigation.kt` merupakan bagian dari package `com.example.scrollablelist`. Baris [3 – 8] berfungsi untuk mengimpor berbagai komponen dari Jetpack Compose yang digunakan untuk mengelola navigasi antar layar. Library ini memungkinkan pembuatan navigasi deklaratif dengan `NavHost`, serta pengaturan rute menggunakan `composable`.

Baris [11 – 13] berfungsi untuk mendefinisikan fungsi `Navigation` yang mengatur alur navigasi pada aplikasi. Di dalamnya, dibuat objek `NavController` untuk mengelola perpindahan antar layar, serta objek `viewModel` yang digunakan untuk menyimpan dan mengambil data item yang sedang dipilih.

Baris [15 – 29] berfungsi untuk mendefinisikan rute navigasi menggunakan `NavHost`, dengan halaman awal ditentukan sebagai `home_Screen`. Rute pertama akan menampilkan `HomeScreen`, yang menerima `NavController` dan `viewModel` sebagai parameter. Rute kedua adalah `detail_Screen`, yang akan menampilkan `DetailScreen`. Sebelum itu, data item yang disimpan di `viewModel` diambil menggunakan `collectAsState`. Jika nilai item tidak null, maka halaman detail akan ditampilkan dengan membawa data item tersebut sebagai parameter.

ItemViewModel.kt

Baris [1] menunjukkan bahwa file ini merupakan bagian dari package `com.example.scrollablelist`. Baris [3 – 7] mengimpor library yang dibutuhkan untuk mendukung navigasi dan pengelolaan tampilan antar layar menggunakan Jetpack Compose.

Baris [9] mendefinisikan fungsi `Navigation()` sebagai komponen tampilan yang bertanggung jawab mengatur alur navigasi aplikasi. Baris [10 – 11] membuat controller navigasi untuk mengelola perpindahan antar layar. Baris [12 – 14] menginisialisasi

ViewModel yang digunakan untuk menyimpan dan mengelola data yang akan dibagikan ke berbagai layar.

Selanjutnya, baris [16 – 22] mendeklarasikan rute navigasi aplikasi menggunakan NavHost. Aplikasi dimulai dari rute home_Screen, dan terdapat dua rute yang masing-masing menampilkan layar utama dan layar detail. Kedua layar ini saling terhubung melalui navigasi yang dikendalikan oleh NavController dan berbagi data melalui ItemViewModel.

ViewModelFactory.kt

Baris [1] menunjukkan bahwa file ini berada dalam package com.example.scrollablelist. Baris [3 – 4] mengimpor kelas-kelas yang dibutuhkan untuk mengelola ViewModel di dalam arsitektur Android, khususnya ViewModel dan ViewModelProvider.

Pada baris [6], didefinisikan sebuah kelas ItemViewModelFactory yang mengimplementasikan ViewModelProvider.Factory. Kelas ini digunakan untuk membuat instance dari ItemViewModel secara manual. Hal ini penting ketika ItemViewModel memerlukan konstruktor khusus atau tidak bisa dibuat langsung oleh sistem.

Baris [7 – 11] merupakan implementasi fungsi create(), yang akan dipanggil saat sistem membutuhkan instance dari ViewModel. Fungsi ini memeriksa apakah kelas yang diminta adalah ItemViewModel. Jika ya, maka instance dari ItemViewModel akan dikembalikan. Jika tidak, maka akan dilempar IllegalArgumentException sebagai penanda bahwa kelas ViewModel yang diminta tidak dikenali.

Penjelasan Step Into, Step Over dan Step Out

- Step into adalah fitur debugging yang berfungsi untuk memasuki fungsi atau metode yang sedang dipanggil. Fitur ini digunakan pada saat kita ingin tahu apa yang terjadi di dalam fungsi tersebut.
- Step Over adalah fitur debugging yang digunakan untuk melewati fungsi dan langsung menjalankan fungsi tersebut tanpa masuk ke dalamnya. Fitur ini digunakan pada saat fungsi yang kita buat sudah benar dan tidak perlu diperiksa detailnya.
- Step Out adalah fitur debugging yang berfungsi untuk keluar dari fungsi saat ini, kemudian kembali ke baris setelah fungsi tersebut dipanggil. Fitur ini digunakan untuk pada saat kita sudah cukup melihat detail dari suatu fungsi dan ingin kembali ke level atas.

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

A. Penjelasan

Pengertian Application Class

Application class adalah class dasar dalam aplikasi Android yang berisi semua komponen lain seperti activity dan layanan. Class Aplikasi, atau Subclass apa pun dari Application class, dibuat sebelum class lain saat proses untuk aplikasi/package yang dibuat. Class ini Fungsi Utama Application Class adalah digunakan untuk inisialisasi status global sebelum status pertama Activity ditampilkan.

Fungsi-fungsi dari Application Class

- Inisialisasi Global

Menjalankan proses inisialisasi satu kali saat aplikasi pertama kali dibuat, seperti konfigurasi library (Firebase, Retrofit, Room, dll.).

- Menyimpan State Global

Menyimpan data atau status aplikasi yang dapat diakses dari berbagai komponen (Activity, Service, dll.).

- Entry Point untuk Dependency Injection

Digunakan sebagai titik masuk untuk sistem dependency injection, seperti Hilt atau Koin.

TAUTAN GIT

Berikut adalah tautan untuk source code yang telah dibuat.

https://github.com/rnddfn/Pemrograman_Mobile